Neural-driven Computing on a Minimal RISC-V ISA

Bernhard Guillon
Advanced Systems Engineering (ASE2026)
Paris Lodron University of Salzburg
Salzburg, Austria
Email: Bernhard.Guillon@stud.plus.ac.at

***Abstract*—This project investigates a lightweight computing environment where perceptron-based neural models are executed as first-class primitives on a minimal RISC-V-like ISA extended with neural/tensor operations.**

***Index Terms*—neural ISA, RISC-V, emulator, custom instructions, lightweight inference**

PROJECT CONTEXT

Supervisor: Univ.-Prof. Dipl.-Inform. Dr.-Ing. Christoph Kirsch
Course: ASE2026 (Advanced Systems Engineering)
University: Paris Lodron University of Salzburg
Date: 2026-03-29

A. Introduction

Modern large language models and AI toolchains typically run inside heavy software stacks (OS + browser + runtime). Many applications will increasingly embed neural components at many layers of the stack. This project explores whether we can design a far more lightweight computing environment that can directly host perceptron-based neural models as first-class computational elements.

We propose to define a minimal instruction set (ISA) with a small set of neural/tensor primitives, implement an emulator for that ISA, and run small trained networks directly in the emulator. The effort covers ISA design, assembler/loader, model-to-ROM/memory serialization, and verification tooling (including automated black-box tests).

B. Vision

Create a compact, demonstrable platform where:

A tiny neural model (e.g., a perceptron or small MLP) can be loaded and executed by a minimal runtime / emulator.

The ISA is RISC-like (RISC-V inspired) and extended with a few neural-tensor primitives (matmul, activation, tensor ops).

I/O is minimal: character input (keyboard-like device) and a simple framebuffer output. The pipeline is: Keyboard input -> neural model inference -> Framebuffer pixel output

The initial demonstration task: train a model that receives short character input and produces a corresponding framebuffer output (e.g., render text or glyphs). The emulator should be capable of loading the model into memory and running inference using the extended ISA.

1) Use of AI: Use LLM to help with research on the ISA, create AI training data. Also write the emulator for the ISA a bootloader and an assembler. We might also need a "transpiler" to transform the model to something which we can load. Or write our own model creator.